

Faza 2: Cuantizare INT8 Weight-Only

Per-Tensor cu Scalare Liniară

ViT-Tiny pe ImageNet-1k validation

Post-Training Quantization — FP32 vs. INT8

Student:

Tudose Alexandru

Grupa 10LF333

An III, Informatică Aplicată

Coordonator:

Conf. dr. ing. Honorius Gâlmeanu

Departamentul de Informatică

Braşov, 2026

Cuprins

1	Introducere	2
2	Metodologie	2
2.1	Schema de cuantizare	2
2.2	Cuantizare selectivă	2
2.3	Configurația experimentală	3
2.4	Metrici de evaluare	3
3	Rezultate	4
3.1	Comparație globală FP32 vs. INT8	4
3.2	Acuratețe și loss	4
3.3	Memorie	4
3.4	Latență	5
3.5	Eroare de cuantizare per strat	5
3.6	Distribuția scale factors	6
3.7	Comparație cross-phase: FP32 / FP16 / INT8	6
3.8	Tradeoff acuratețe–eficiență	7
4	Analiza Outlierilor — Block 7	7
5	Discuție	8
5.1	Validarea cuantizării selective	8
5.2	Weight-only vs. activation quantization	8
5.3	Comparație cu Faza 0	8
5.4	Limitări	8
6	Concluzii	9

1 Introducere

Acest raport documentează **Faza 2** a lucrării de licență, în care evaluăm cuantizarea **INT8 weight-only** cu scalare liniară per-tensor pe ViT-Tiny pretrained. Spre deosebire de Faza 1 (FP16), care reduce precizia la 16 biți, INT8 comprimă ponderile la doar **8 biți**, oferind o reducere teoretică de $4\times$ a memoriei.

Obiective:

1. Implementarea cuantizării INT8 cu scalare liniară per-tensor pentru straturile `nn.Linear` ale ViT-Tiny.
2. Evaluarea impactului asupra acurateții, latenței și memoriei față de baseline-ul FP32 stabilit în Faza 1.
3. Analiza erorii de cuantizare per strat (MSE, MAE, scale factor) pentru identificarea straturilor problematice.
4. Comparația cross-phase FP32 / FP16 / INT8.

Context: Cuantizarea weight-only INT8 stochează ponderile ca `int8` (1 byte/param) dar efectuează inferența în `float32` prin dequantizare la fiecare forward pass. Aceasta reduce memoria, dar *nu* accelerează inferența — overhead-ul de dequantizare poate chiar încetini modelul.

2 Metodologie

2.1 Schema de cuantizare

Cuantizarea utilizează scalare liniară per-tensor, cu formulele:

$$\text{scale} = \frac{\max(|W|)}{127} \quad (1)$$

$$q = \text{round}\left(\frac{W}{\text{scale}}\right).\text{clamp}(-128, 127).\text{to}(\text{int8}) \quad (2)$$

$$\hat{W} = q.\text{float}() \times \text{scale} \quad (3)$$

Ecuția (3) este aplicată la fiecare forward pass: ponderile INT8 sunt dequantizate la `float32` înainte de multiplicarea matriceală. Aceasta garantează o mapare bijectivă la 256 valori discrete, spre deosebire de formatul `float8_e4m3fn` testat în Faza 0.

2.2 Cuantizare selectivă

Nu toate straturile sunt cuantizate. Straturile sensibile la erori numerice sunt excluse:

Tabela 1: Straturi cuantizate vs. exclude

Categorie	Straturi	Motiv
Cuantizate	attn.qkv, attn.proj mlp.fc1, mlp.fc2	Proiecții liniare — robuste la cuantizare MLP layers — cea mai mare parte din parametri
Exclude (FP32)	norm1, norm2, norm cls_token, pos_embed head	LayerNorm — sensibil la erori de scalare Parametri de embedding — critici pentru clasificare Classification head — stratul final de decizie

Rezultat: 48 din 49 straturi `nn.Linear` sunt cuantizate (5 308 416 parametri). Doar head (192 000 parametri) rămâne în FP32.

2.3 Configurația experimentală

Tabela 2: Configurația experimentală

Parametru	Valoare
Model	vit_tiny_patch16_224 (timm)
Parametri	5.7 milioane (din care 5.3M cuantizate)
Pretrained	ImageNet-1k (pretrained=True)
Dataset evaluare	ImageNet-1k validation (50 000 imagini, 1 000 clase)
Batch size	64
Device	Apple M4, backend MPS
PyTorch	2.9.1
Metoda cuantizare	Per-tensor INT8, scalare liniară
Granularitate	Un factor de scalare per matrice de ponderi

2.4 Metrici de evaluare

Pe lângă metricile standard (accuracy, loss, latency, memory), Faza 2 introduce metrice specifice cuantizării:

- **MSE** (Mean Squared Error): $\frac{1}{n} \sum (W - \hat{W})^2$ — eroarea medie de reconstrucție per strat.
- **MAE** (Mean Absolute Error): $\frac{1}{n} \sum |W - \hat{W}|$ — eroarea absolută medie.
- **Scale factor:** Valoarea $s = \max(|W|)/127$ per strat — reflectă dinamicul ponderilor.

3 Rezultate

3.1 Comparație globală FP32 vs. INT8

Tabela 3: Rezultate FP32 vs. INT8 pe ImageNet-1k validation

Format	Accuracy (%)	Loss	Latency (ms)	Memory (MB)
FP32 (baseline)	75.456	0.9420	200.99	21.81
INT8 (per-tensor)	75.134	0.9563	217.09	6.62
Diferență	−0.322 pp	+0.014	0.93× (mai lent)	3.29× reducere

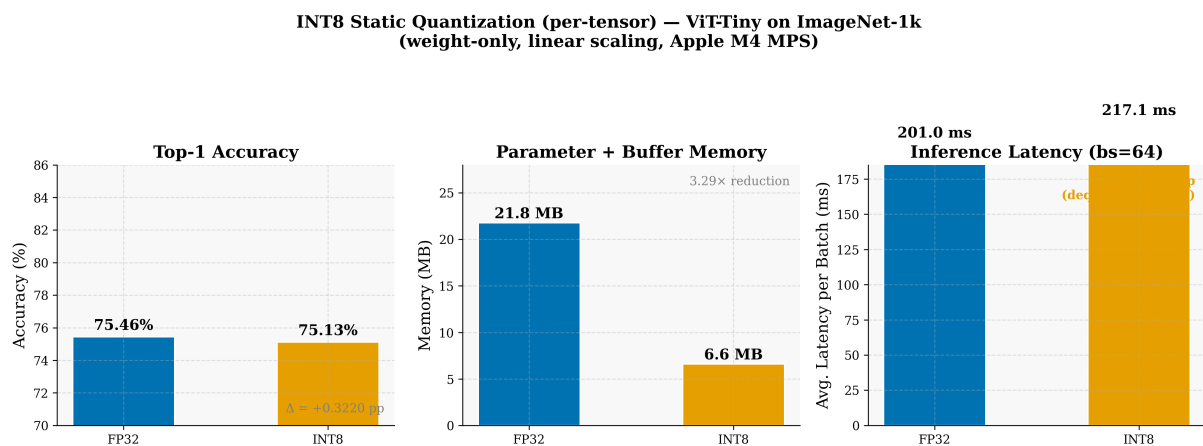


Figura 1: Overview Faza 2: acuratețe, memorie și latență — FP32 vs. INT8 per-tensor. Degradarea de acuratețe este mică (−0.322 pp), reducerea de memorie este semnificativă (3.29×), dar nu există speedup (overhead de dequantizare).

3.2 Acuratețe și loss

Degradarea de −0.322 pp (de la 75.456% la 75.134%) este acceptabilă pentru o reducere de memorie de 3.29×. Aceasta reprezintă 161 imagini din 50 000 clasificate diferit.

Loss-ul crește de la 0.9420 la 0.9563 (+1.5%), indicând o ușoară “estompare” a distribuției softmax cauzată de eroarea de cuantizare. Totuși, această creștere nu se traduce într-o degradare practică semnificativă a acurateții.

3.3 Memorie

Reducerea de 3.29× (de la 21.81 MB la 6.62 MB) este explicată astfel:

- 48 straturi cuantizate: $5\,308\,416 \times 1 \text{ byte} = 5.06 \text{ MB}$ (față de $5\,308\,416 \times 4 \text{ bytes} = 20.25 \text{ MB}$ în FP32)
- Straturi excluse (head, bias-uri): rămân în FP32
- Buffer-uri suplimentare: scale factors (48 valori float32) — overhead neglijabil
- Reducerea nu este exact 4× deoarece nu toate componentele sunt cuantizate

3.4 Latență

INT8 este mai lent decât FP32 cu un factor de $0.93\times$ (217.09 ms vs. 200.99 ms). Aceasta este o consecință directă a cuantizării weight-only:

1. La fiecare forward pass, ponderile `int8` sunt dequantizate la `float32` ($\hat{W} = q \cdot s$).
2. Multiplicarea matriceală se execută în `float32`, ca la modelul original.
3. Overhead-ul de dequantizare (≈ 16 ms/batch) anulează orice beneficiu de bandwidth.

Concluzie: Pentru a obține speedup real cu INT8, ar fi necesară inferență nativă în aritmetică pe numere întregi (e.g., NVIDIA INT8 Tensor Cores sau `torch.int8` GEMM kernels), care nu sunt disponibile pe Apple M4 MPS.

3.5 Eroare de cuantizare per strat

Tabela 4: Statistici sumare ale erorii de cuantizare

Metrică	Valoare
Straturi cuantizate	48
Parametri cuantizați	5 308 416
MSE mediu per strat	2.88×10^{-6}
Worst layer (MSE)	<code>blocks.7.mlp.fc2</code> (3.54×10^{-5})
Best layer (MSE)	<code>blocks.8.attn.proj</code> (2.69×10^{-7})
Raport worst/best	131.6×

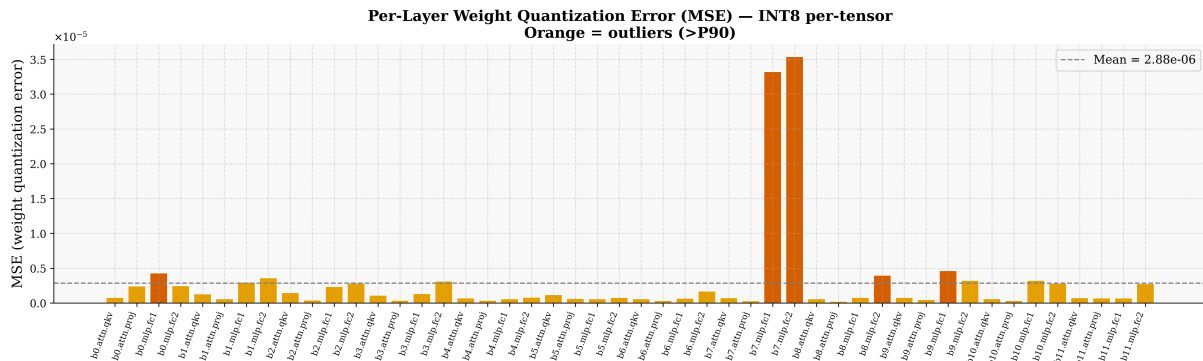


Figura 2: MSE per strat cuantizat. Straturile `blocks.7.mlp.fc1` și `blocks.7.mlp.fc2` sunt outlieri evidenti (MSE $\approx 12\times$ mai mare decât media), marcate cu portocaliu (>P90). Aceste straturi au scale factors de ≈ 0.020 , indicând ponderi cu valori absolute mult mai mari decât restul modelului.

3.6 Distribuția scale factors

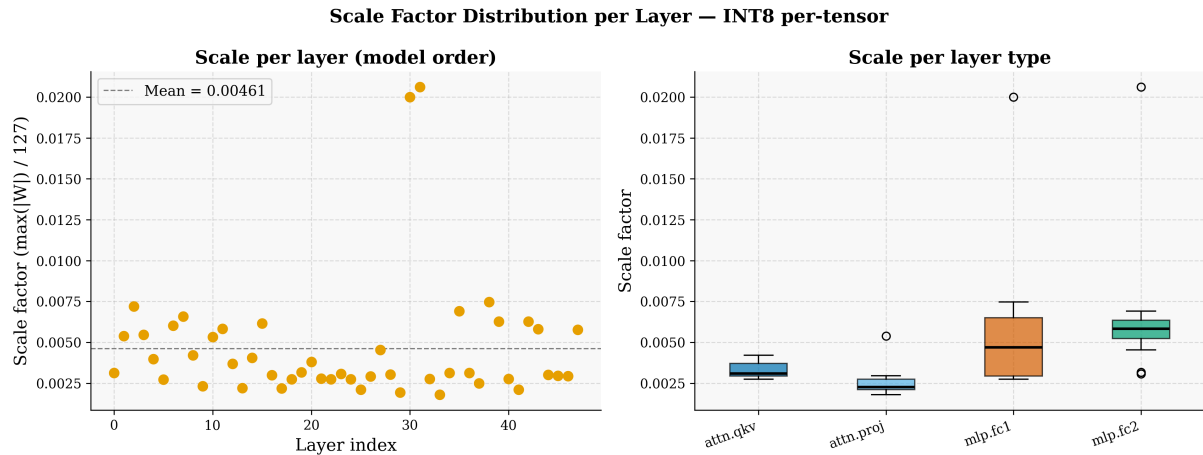


Figura 3: Distribuția scale factors per strat. (Stânga) Valori per strat în ordinea modelului — se observă spike-urile la block 7. (Dreapta) Boxplot per tip de strat: `attn.qkv` și `mlp.fc1/fc2` au variabilitate mai mare decât `attn.proj`.

Outlierii din block 7 au scale factors de ≈ 0.020 , comparativ cu media de ≈ 0.004 . Aceasta înseamnă că ponderile acestor straturi au valori absolute de $5\times$ mai mari decât media, ceea ce forțează o “grilă” de cuantizare mult mai grosolană (fiecare nivel INT8 acoperă un interval mai larg).

Implicație: Aceste straturi sunt candidați ideali pentru cuantizare *per-channel* (un scale factor per rând), care va fi evaluată în Faza 3.

3.7 Comparație cross-phase: FP32 / FP16 / INT8

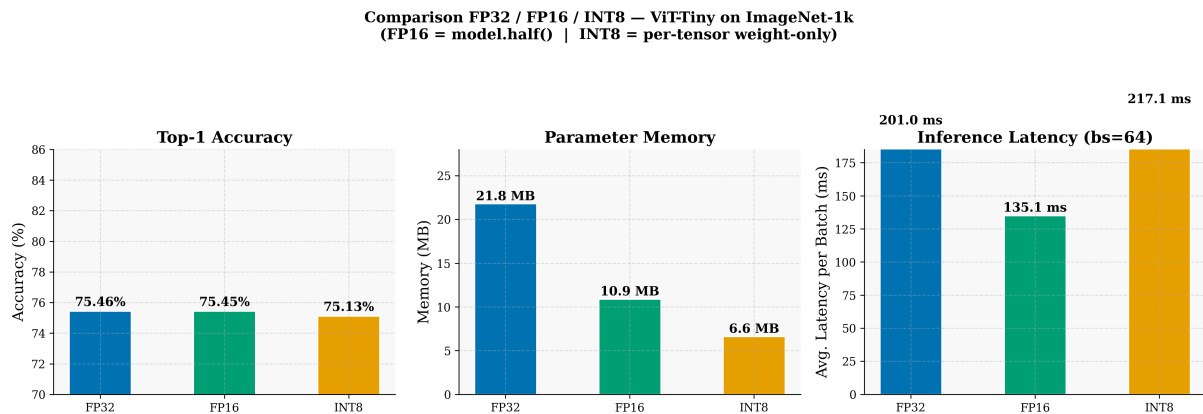


Figura 4: Comparație între cele trei formate evaluate până acum. FP16 oferă cel mai bun raport acuratețe/eficiență (zero degradare, $2\times$ memorie, $1.22\times$ speedup). INT8 reduce memoria mai agresiv ($3.29\times$), dar cu o ușoară degradare și fără speedup.

Tabela 5: Comparație cumulativă Faza 1 + Faza 2

Format	Accuracy	Degradare	Memorie	Speedup
FP32	75.456%	—	21.81 MB	1.00×
FP16 (Faza 1)	75.452%	−0.004 pp	10.91 MB (2.00×	1.22×
INT8 (Faza 2)	75.134%	−0.322 pp	6.62 MB (3.29×	0.93×

3.8 Tradeoff acuratețe–eficiență

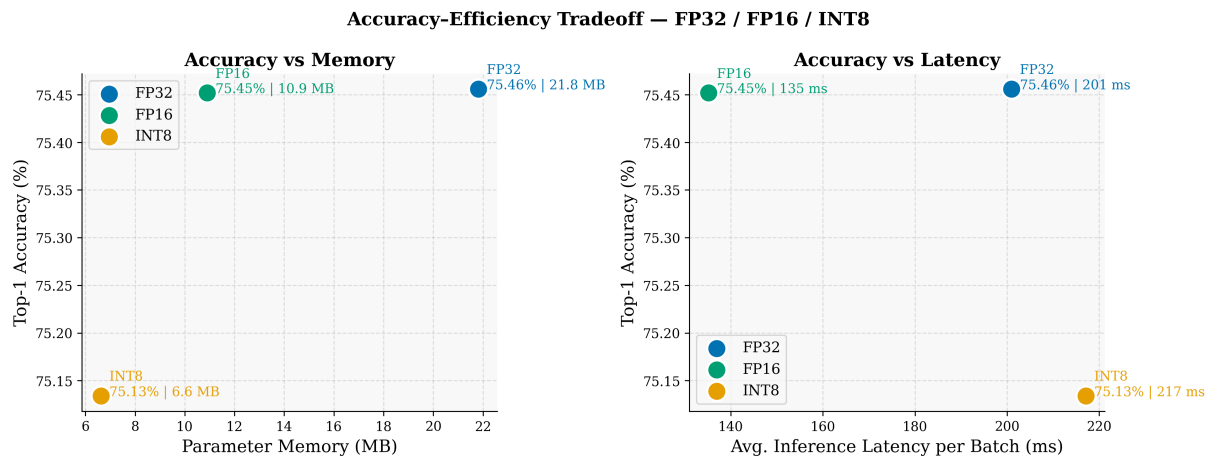


Figura 5: Scatter plot acuratețe vs. memorie (stânga) și acuratețe vs. latență (dreapta). FP16 domină pe ambele axe. INT8 oferă o reducere de memorie suplimentară față de FP16, dar cu un cost în latență.

4 Analiza Outlierilor — Block 7

Straturile `blocks.7.mlp.fc1` și `blocks.7.mlp.fc2` prezintă un comportament atipic:

Tabela 6: Outlieri vs. media modelului

Strat	MSE	MAE	Scale	Raport MSE/medie
Media model	2.88×10^{-6}	—	0.004	1.0×
b7.mlp.fc1	3.32×10^{-5}	0.0050	0.020	11.5×
b7.mlp.fc2	3.54×10^{-5}	0.0052	0.021	12.3×

Cauza: Aceste straturi conțin ponderi cu valori absolute semnificativ mai mari ($\max |W| \approx 2.5$, comparativ cu ≈ 0.5 pentru straturile tipice). Cuantizarea per-tensor alocă un singur scale factor pentru întreaga matrice, ceea ce înseamnă că valorile mici sunt cuantizate cu o rezoluție foarte scăzută.

Soluție: Cuantizarea *per-channel* (un scale factor per rând al matricii) va fi evaluată în Faza 3. Aceasta permite fiecărui canal de ieșire să aibă propriul factor de scalare, adaptându-se la dinamicul local al ponderilor.

5 Discuție

5.1 Validarea cuantizării selective

Strategia de a exclude LayerNorm, cls_token, pos_embed și head s-a dovedit corectă: degradarea de doar -0.322 pp pentru 48 straturi cuantizate confirmă că straturile liniare din Attention și MLP sunt robuste la cuantizare INT8.

5.2 Weight-only vs. activation quantization

Implementarea noastră cuantizează doar *ponderile*. Activările rămân în float32 pe tot parcursul inferenței. Aceasta:

- **Avantaj:** Simplitate — nu necesită calibrare pe date de referință, nu depinde de distribuția intrărilor.
- **Dezavantaj:** Nu oferă speedup pe niciun hardware, deoarece GEMM-urile rămân în float32.

Pentru speedup real ar fi necesară cuantizarea *completă* (ponderi + activări), cu suport hardware nativ (e.g., NVIDIA int8 GEMM via TensorRT sau ONNX Runtime).

5.3 Comparație cu Faza 0

Tabela 7: Progresul cuantizării: Faza 0 vs. Faza 2

Aspect	Faza 0 (CIFAR-10)	Faza 2 (ImageNet-1k)
Format	FP8 E4M3FN	INT8 scalare liniară
Dataset	10 000 imagini, 10 cls	50 000 imagini, 1 000 cls
Cuantizare	Toate straturile	Selectivă (skip norm/head)
Mapare	Non-bijectivă	Bijectivă (256 valori)
Degradare	< 0.02 pp	-0.322 pp
Analiză per-strat	Nu	Da (MSE, MAE, scale)

Degradarea mai mare pe ImageNet (-0.322 pp vs. < 0.02 pp pe CIFAR-10) confirmă ipoteza din Faza 0: **dataset-urile complexe sunt mai sensibile la erori de cuantizare** decât dataset-urile simple.

5.4 Limitări

1. **Granularitate per-tensor:** Un singur scale factor per matrice nu se adaptează la distribuții non-uniforme ale ponderilor. Faza 3 va evalua alternativa per-channel.
2. **Fără speedup:** Dequantizarea la float32 la fiecare forward pass introduce overhead. Aceasta este o limitare fundamentală a cuantizării weight-only pe hardware fără suport INT8 nativ.
3. **Hardware-specific:** Rezultatele de latență sunt specifice Apple M4 MPS. Pe GPU-uri NVIDIA cu INT8 Tensor Cores, profilul de latență ar fi diferit.

4. **Un singur model:** Doar ViT-Tiny a fost evaluat. Modele mai mari pot prezenta pattern-uri diferite de distribuție a ponderilor.

6 Concluzii

1. **Degradare acceptabilă:** INT8 per-tensor introduce o pierdere de -0.322 pp (161 imagini din 50 000), acceptabilă pentru o reducere semnificativă de memorie.
2. **Reducere de memorie $3.29\times$:** De la 21.81 MB la 6.62 MB. Nu exact $4\times$ teoretic, deoarece bias-urile și head-ul rămân în FP32.
3. **Fără speedup pe MPS:** INT8 weight-only este chiar **7%** mai lent decât FP32, din cauza overhead-ului de dequantizare la fiecare forward pass.
4. **Outlieri identificați:** Straturile `blocks.7.mlp.*` au MSE de $\approx 12\times$ media, cu scale factors de $5\times$ mai mari. Acestea sunt candidații principali pentru cuantizare per-channel.
5. **Cuantizare selectivă validată:** Excluderea LayerNorm, cls_token, pos_embed și head menține acuratețea la un nivel acceptabil, confirmând robustețea straturilor liniare din Attention și MLP.
6. **Direcții Faza 3:** Analiza de sensibilitate per strat (câte un strat cuantizat la un moment dat), comparația per-tensor vs. per-channel, și profilarea detaliată a memoriei și latenței.

Bibliografie

- [1] A. Dosovitskiy, L. Beyer, A. Kolesnikov et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” *ICLR*, 2021. [arXiv:2010.11929](#).
- [2] B. Jacob, S. Kligys, B. Chen et al., “Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference,” *CVPR*, 2018. [arXiv:1712.05877](#).
- [3] M. Nagel, M. Fournarakis, R. A. Amjad, et al., “A White Paper on Neural Network Quantization,” 2021. [arXiv:2106.08295](#).
- [4] T. Dettmers, M. Lewis, Y. Belkada, L. Zettlemoyer, “LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale,” *NeurIPS*, 2022. [arXiv:2208.07339](#).
- [5] R. Wightman, “PyTorch Image Models (timm),” 2019. <https://github.com/rwightman/pytorch-image-models>.
- [6] O. Russakovsky, J. Deng, H. Su et al., “ImageNet Large Scale Visual Recognition Challenge,” *IJCV*, vol. 115, no. 3, pp. 211–252, 2015. [arXiv:1409.0575](#).